

Interactive Embedded Synthesizer

Embedded Systems

2025

By Diogo Araújo, Rodrigo Araújo and Ricardo Oliveira

Contents



1 Overview	3
1.a Project Summary	4
1.b Bill of Materials	5
2 Requirements	6
2.a Functional Requirements	7
2.b Non-Functional Requirements	9
3 System Modeling	10
3.a Sensor-to-Parameter Mapping	11
3.b Oscillator and LFO Models	13
3.c Envelope and Filter Models	14
4 Architecture	15
4.a Hardware Circuit	16
4.b Software Architecture	18
4.c Audio Engine	19
5 Conclusion	20
5.a Summary	21

1 Overview

Project Summary



This project proposes an **interactive embedded synthesizer** designed as a distributed system with three main components:

Arduino

Physical control interface — reads sensors, knobs, and buttons

Raspberry Pi

Audio synthesis engine — generates and streams sound in real time

Android Device

Mobile client — handles audio playback and oscilloscope visualization

The main objective is to transform physical gestures into real-time sound generation, transmitting synthesis parameters continuously across the distributed pipeline.

Bill of Materials



Total component cost: **€54.44**

Component	Cost
Ultrasonic proximity sensor	€1.89
Joystick control	€0.34
Five potentiometers	€0.55
Breadboards and cables	€3.80
6 Buttons	€2.63
Arduino	€3.46
Raspberry Pi 4	€39.99
LCD screen	€1.78

2 Requirements

Functional Requirements



Key mappings from physical controls to synthesis parameters:

Ultrasonic sensor

Controls LFO frequency

Potentiometers (×4)

Parameterize ADSR envelope — Attack, Decay, Sustain, Release

LFO potentiometer

Controls modulation depth

Waveform button

Cycles waveform — Sine → Saw → Square → Triangle → Noise

Functional Requirements (ii)



Joystick X-axis

Selects filter mode (LPF / Neutral / HPF) and cutoff frequency

Joystick Y-axis

Modulates pitch ± 2 semitones

Five chord buttons

Each mapped to a musical chord

LCD

Displays active waveform

Non-Functional Requirements



Constraint	Target
End-to-end latency	< 50 ms
Control update rate	≥ 50 Hz (every 20 ms)
Audio sample rate	48 kHz, uninterrupted
Control message jitter	< 10 ms
CPU usage (Raspberry Pi)	$\leq 70\%$ under normal load
Concurrent parameter updates	No playback restart required
Responsibility separation	Arduino: sensing; RPi: synthesis; Android: playback

3 System Modeling

Sensor-to-Parameter Mapping



The system maps raw hardware readings to normalized synthesis control variables:

Ultrasonic sensor

Normalized distance linearly maps to LFO frequency within a defined operating range

Joystick X-axis

Magnitude determines filter cutoff; sign selects LPF or HPF

Joystick Y-axis

Displacement maps to a pitch offset of up to ± 2 semitones

Potentiometers

Each 10-bit reading is normalized to a unit interval and mapped linearly to its corresponding ADSR or LFO depth parameter

Sensor-to-Parameter Mapping (ii)



Chord buttons

Each button maps to a fixed musical chord via equal-tempered frequency conversion

All sensor values are normalized before being forwarded to the synthesis engine, ensuring consistent parameter ranges regardless of hardware variation.

Oscillator and LFO Models



Supported waveforms, selectable via the waveform button:

- **Sine** — smooth, pure tone
- **Square** — hollow, buzzy character
- **Sawtooth** — bright, harmonically rich
- **Triangle** — softer than square, flute-like

Low-Frequency Oscillator modulates the effective oscillator frequency each sample. The ultrasonic sensor controls modulation speed; the dedicated potentiometer controls modulation depth. Together they produce vibrato-style pitch variation.

Envelope and Filter Models



ADSR Envelope — five-stage amplitude shaping triggered by button press and release:

- **Attack:** linear ramp from silence to full amplitude
- **Decay:** ramp down from full amplitude to the sustain level
- **Sustain:** held constant while the button is held
- **Release:** fades to silence after the button is released

Filter — joystick X-axis selects mode and continuously controls cutoff frequency:

- Left of center applies **low-pass filtering** — attenuates high frequencies
- Right of center applies **high-pass filtering** — attenuates low frequencies
- Centered position leaves the signal **unfiltered**

4 Architecture

Hardware Circuit



The circuit is organized into **three modules**:

Keypad module

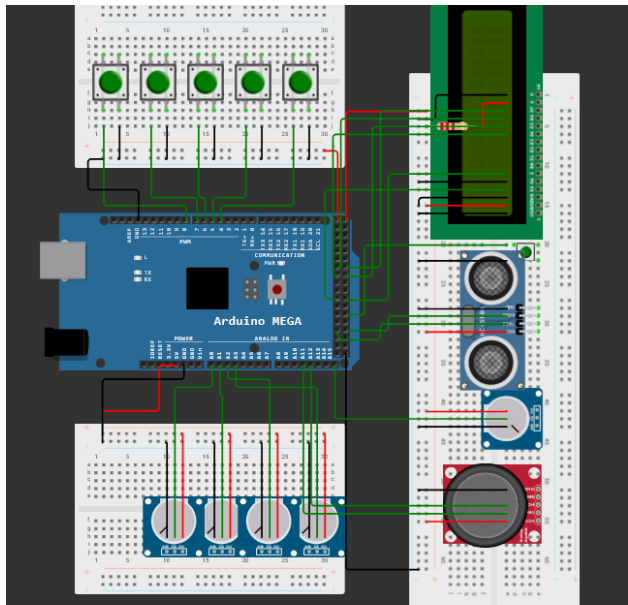
Five buttons in INPUT_PULLUP mode, each mapped to a musical chord

ADSR controller

Four potentiometers assigned to Attack, Decay, Sustain, Release

LFO controller

Joystick + ultrasonic sensor + potentiometer + waveform button + LCD display





Hardware Circuit (ii)

All modules connect to an **Arduino Mega**, which polls inputs, applies debouncing (5 ms), and transmits a **Packet** struct over UART at 115200 baud every 5 ms with an XOR checksum.

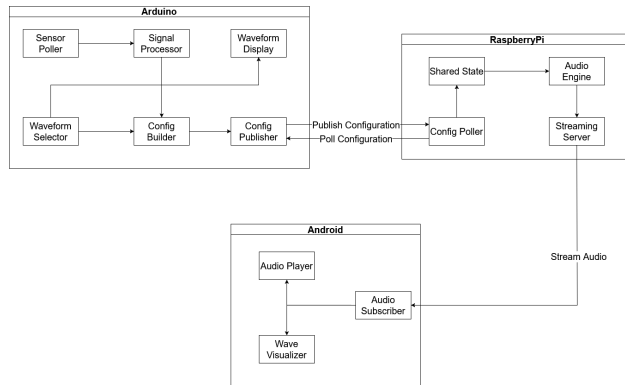
The **Packet** struct is transmitted over UART at 115200 baud every 5 ms, carrying all sensor and control data in a compact, packed binary format verified by an XOR checksum.

```
typedef struct __attribute__((packed)) {  
    uint8_t  start;  
    uint8_t  waveform;  
    int16_t  potentiometers[5];  
    int16_t  joystick[2];  
    float    ultrasonic;  
    uint8_t  buttons[5];  
    uint8_t  checksum;  
    uint8_t  end;  
} Packet;
```

Software Architecture



Layer	Responsibilities
Arduino	Poll sensors, debounce buttons, manage waveform FSM, transmit via UART
Raspberry Pi	Receive packets, update atomic state, run node-graph synthesis, stream via UDP
Android	UDP handshake, receive PCM frames, playback at 48 kHz, oscilloscope visualization

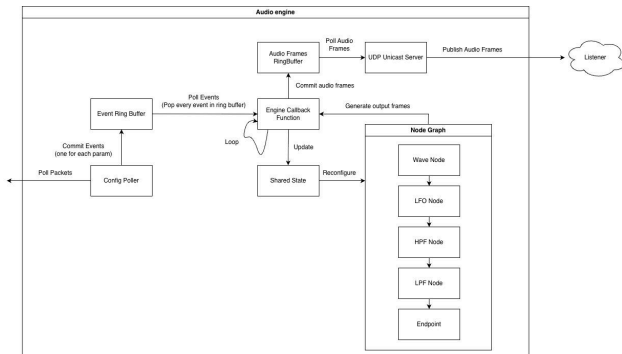


Audio Engine

Audio is produced through a fixed **node graph** processing chain:

Polyphonic Oscillator → **Tremolo (LFO)** → **HPF** → **LPF** → **Output**

- All parameters stored in a lock-free atomic shared state
- An event ring buffer mediates between the control and audio threads
- Per-chord ADSR envelope runs as a five-state machine: Idle → Attack → Decay → Sustain → Release
- A UDP thread forwards output frames to the Android client



5 Conclusion

Summary



This project specifies an **interactive embedded synthesizer** distributed across three hardware layers:

- **Physical interaction** via Arduino sensors drives synthesis parameters in real time
- **Audio synthesis** on the Raspberry Pi uses a polyphonic node graph with ADSR, LFO, and filtering
- **Mobile client** on Android provides live audio playback and oscilloscope visualization

Key embedded systems concepts explored:

Latency

End-to-end target below 50 ms

Concurrency

Lock-free atomic state shared between communication and audio threads

Real-time processing

48 kHz uninterrupted synthesis on constrained hardware

Summary (ii)



Distributed communication

UART (Arduino→RPi) and UDP (RPi→Android)

Future work could extend synthesis techniques, add richer effects, and expand mobile visualization.